Advanced Computer-Aided VLSI System Design

Final Project:
Multi-Core GEMM Accelerator

Lecturer: Yueh-Feng Tsai

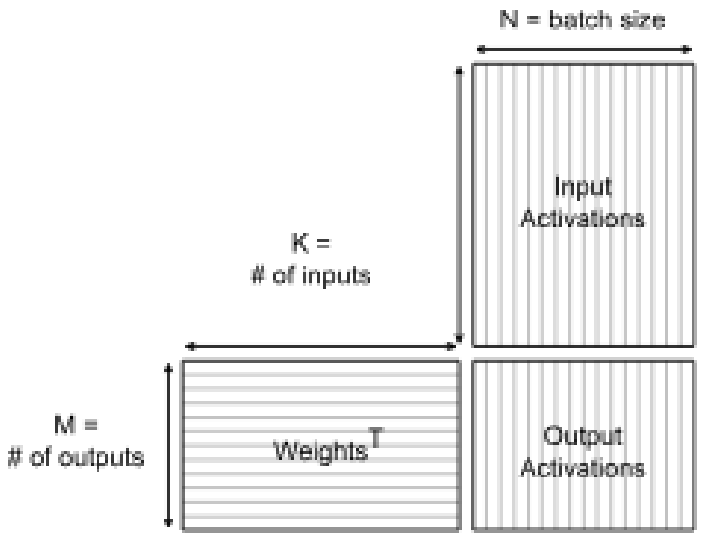
Graduate Institute of Electronics Engineering, National Taiwan University



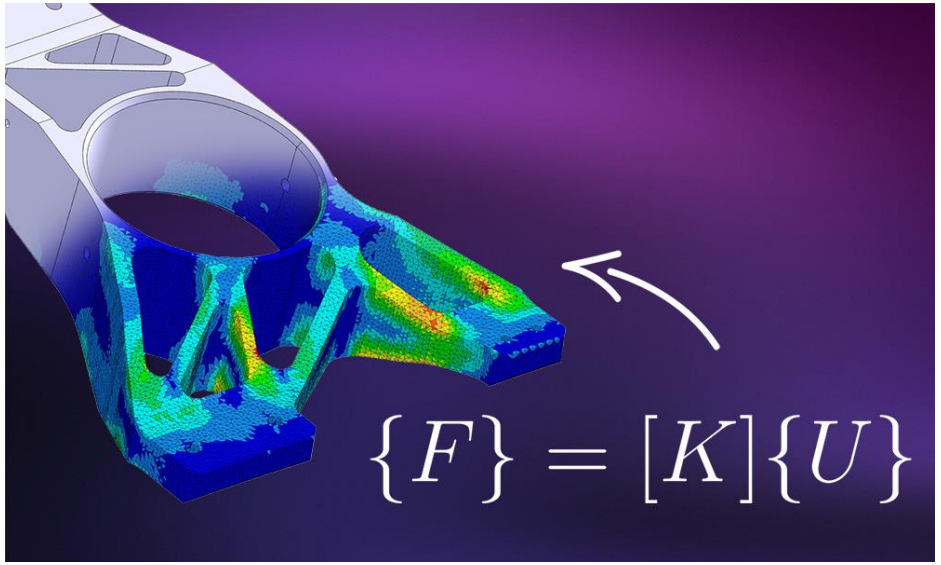
NTU GIEE

Overview

- General matrix-matrix multiplication (GEMM) is an essential operation in **machine learning (ML)** and **scientific computing**
  - **Machine learning**: $\mathrm{OF} = \mathrm{IF} \times \mathrm{Weight}$
  - **Scientific computing**: $\mathrm{F} = \mathrm{K} \times \mathrm{U}$



Linear Layer in ML [1]



Finite Element Analysis in Scientific Computing [2]

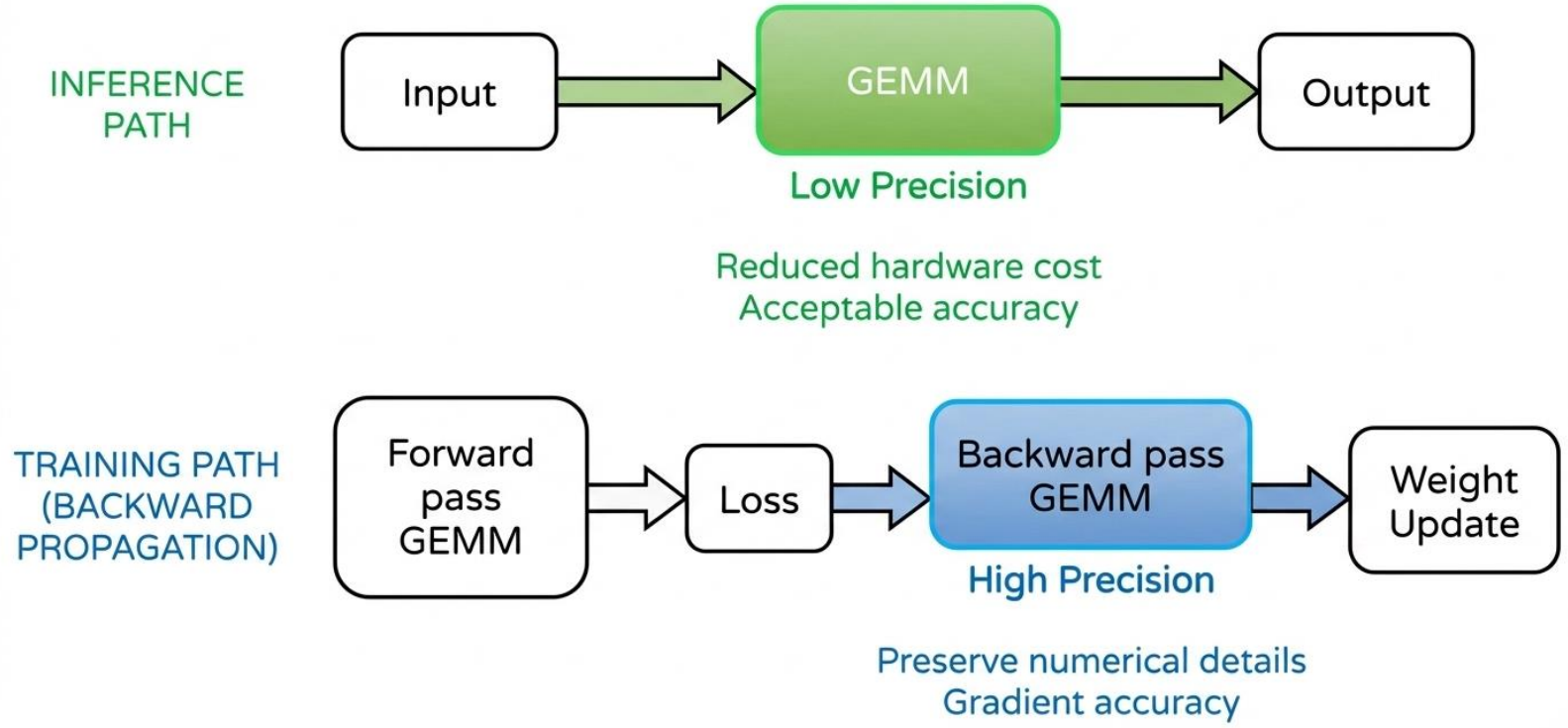
GEMM Formulation

- A general formulation of GEMM can be written as $\boldsymbol{Z} = \boldsymbol{X} \times \boldsymbol{Y}$
  - $\boldsymbol{X} \in \mathbb{R}^{\boldsymbol{N}\times\boldsymbol{K}}$
  - $\boldsymbol{Y} \in \mathbb{R}^{\boldsymbol{K}\times\boldsymbol{M}}$ **(It will be represented in $\boldsymbol{Y}$ transpose in following, $\boldsymbol{Y^T}$ *or* $\boldsymbol{YT}$)**
  - $\boldsymbol{Z} \in \mathbb{R}^{\boldsymbol{N}\times\boldsymbol{M}}$
- Each element of $\boldsymbol{Z}$: $\boldsymbol{Z}[i][j] = \sum(\boldsymbol{X}[i][k] \times \boldsymbol{YT}[j][k])$

$$\begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} \bullet \begin{bmatrix} YT_{11} & YT_{12} \\ YT_{21} & YT_{22} \end{bmatrix} = \begin{bmatrix} Z_{11} & Z_{12} \\ Z_{21} & Z_{22} \end{bmatrix}$$

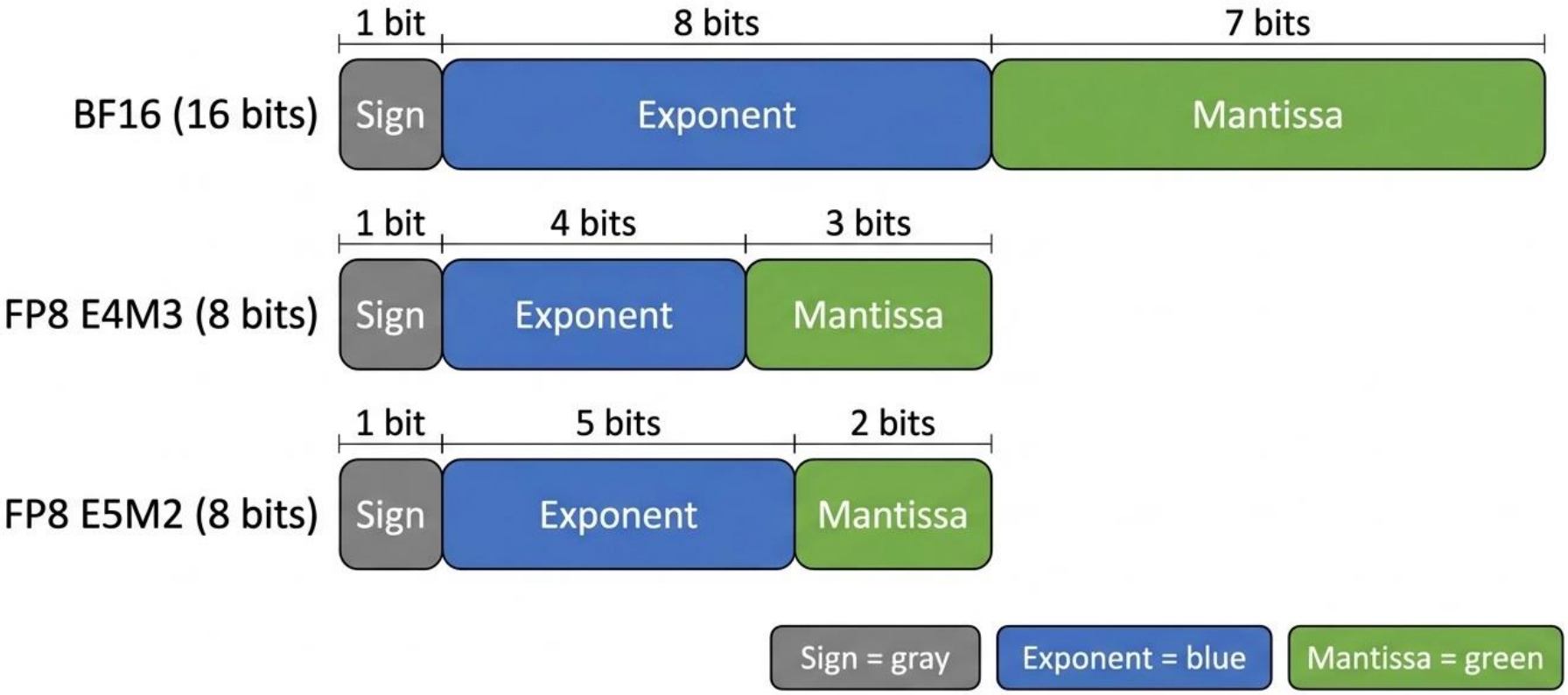
Multiple Precision

- Lower precision formats reduce hardware cost with acceptable accuracy for ML inference
- High-precision formats is utilized to preserve details for backward propagation in ML and scientific computing



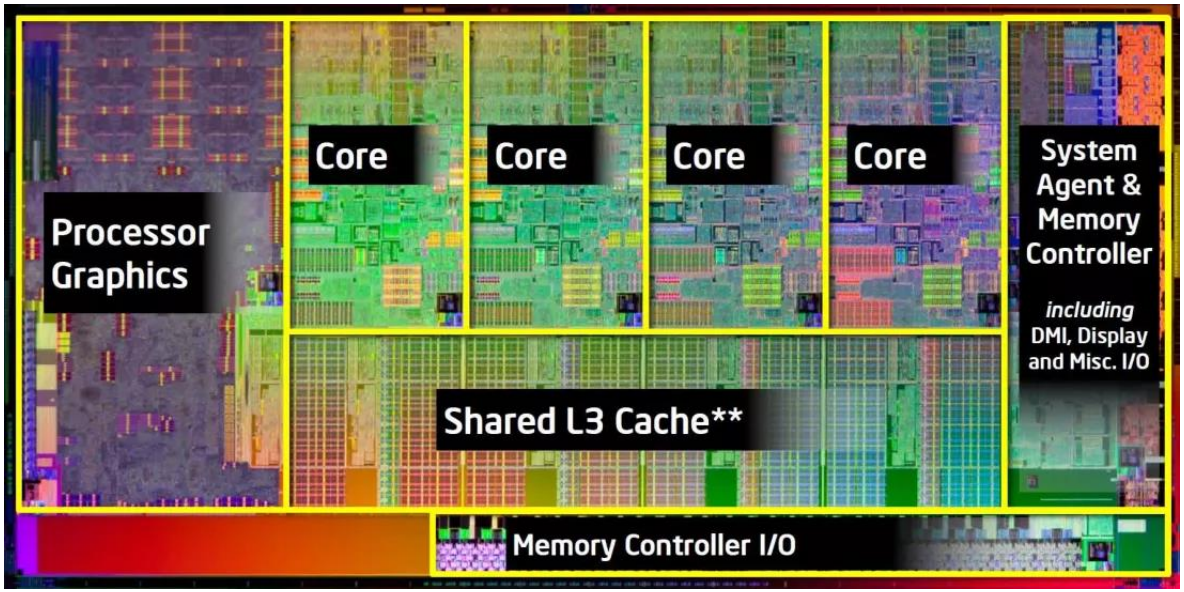
Multiple Precision

- Bits of each part in floating-point format can be adjusted
- Supported format: **BF16, FP8-E4M3, FP8-E5M2**



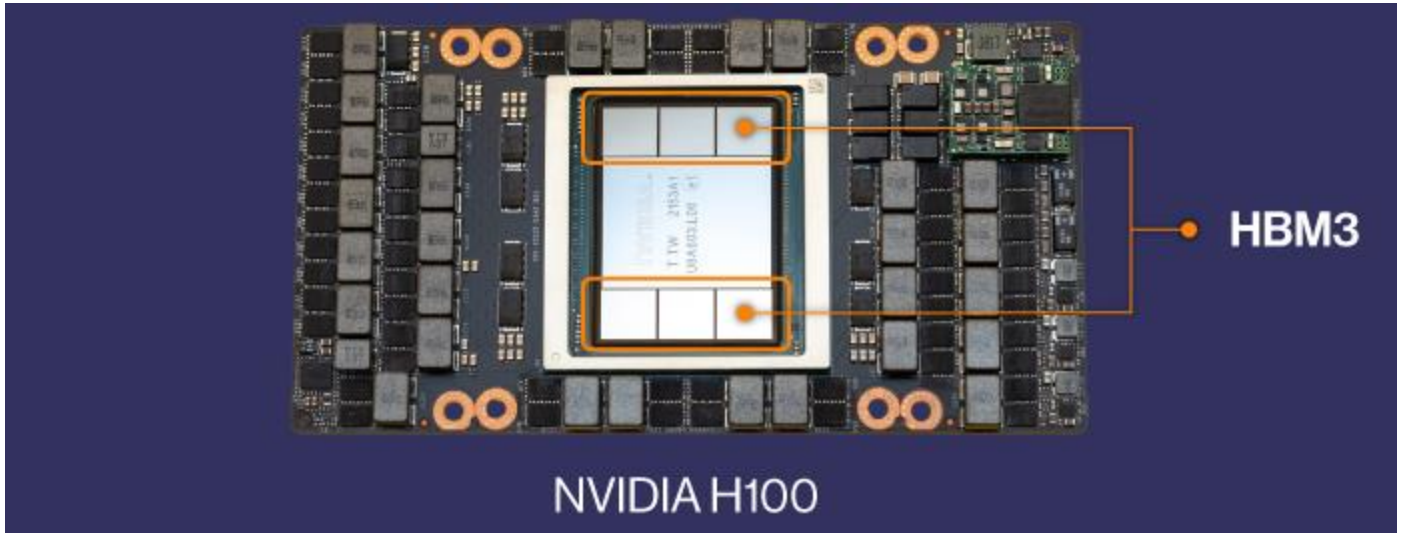
Multi-Core Accelerator

- Each GEMM core operates independently, allowing flexible scaling from single-core to multi-core configurations for different application
- Identical core design enables a "design-once, verify-once" methodology. Scaling up only requires verifying the interconnect, not redesigning the compute logic



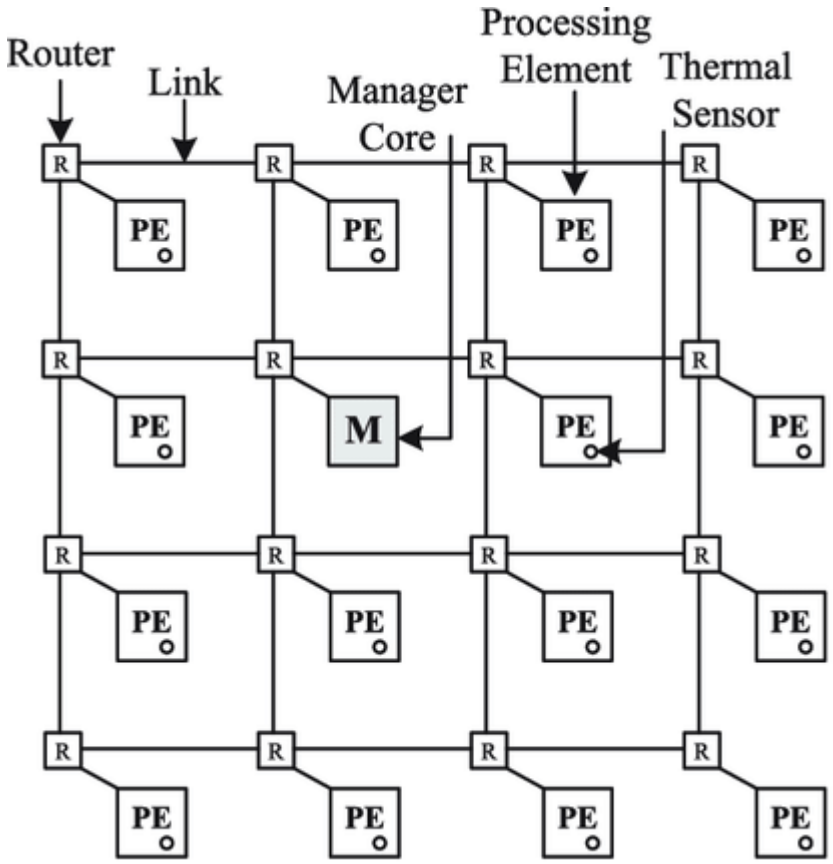
Distributed Memory

- Conventional approach: all cores connected to shared memory
  - High routing source required
  - Less scalability
- Advanced approach: multiple memories are integrated in the system
  - Multiple DRAM/HBM connected to the computation die

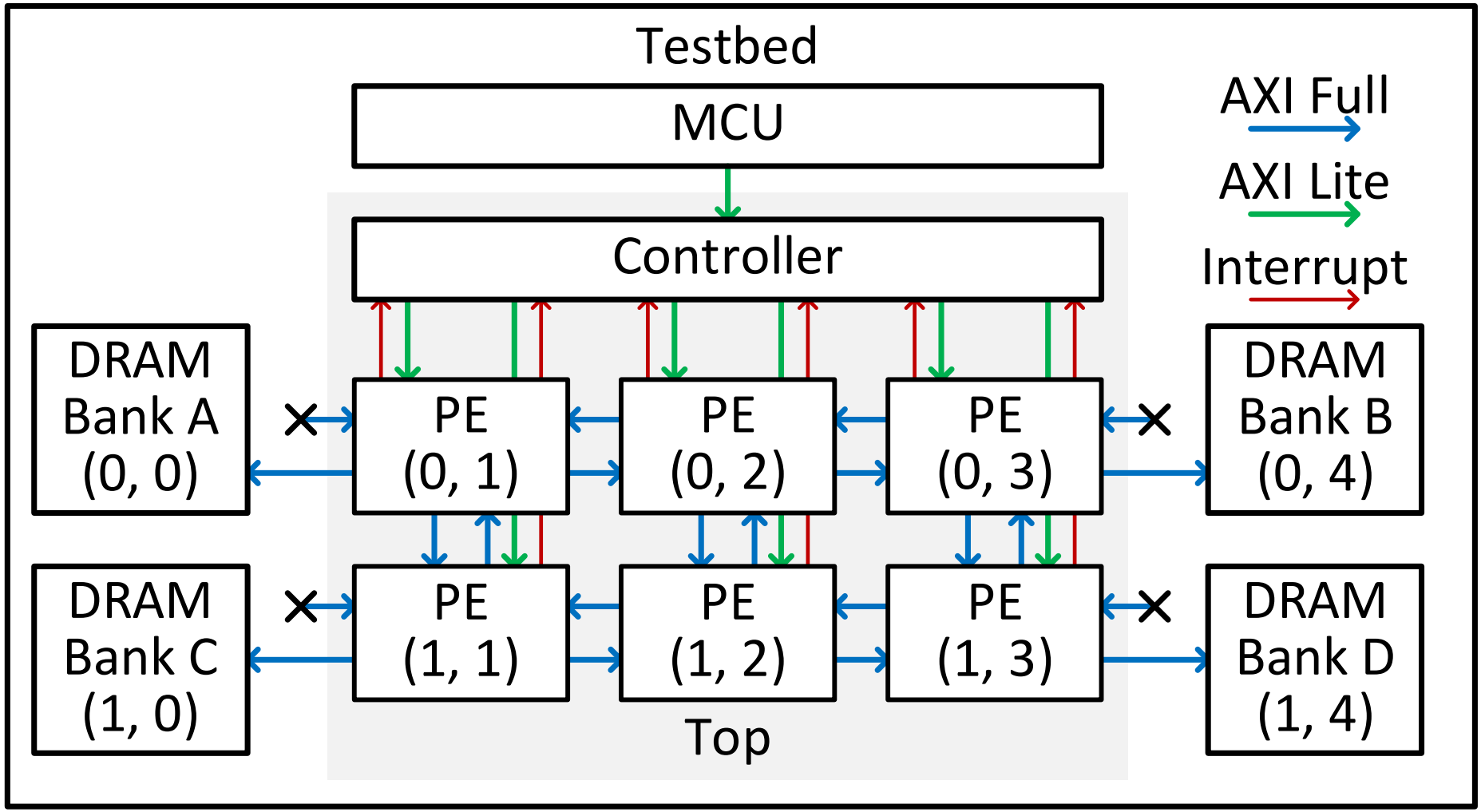


Network on Chip

- Network on chip (NoC) provides regular, scalable interconnect with predictable latency and modular routing



Architecture



Architecture

- **PE:** Each process element (PE) performs independent matrix multiply-accumulate operations
- **Controller:** The controller manages data distribution and task scheduling across all PEs.
- **MCU:** The microcontroller unit (MCU) handles top-level system control, configuration, and communication with the external host via AXI Lite interface
- **DRAM:** Distributed memory nodes placed at the edges of the mesh

Top I/O: Clock and Reset

Signal Name	I/O	Width	Description
`clk_pe`	I	1	The system clock signal for PEs You can define the clock frequency (>= 100MHz)
`clk_data_bus`	I	1	The system clock signal for all AXI-full buses The frequency is fixed at **200MHz (clock period 5ns)**
`clk_ctrl_bus`	I	1	The system clock signal for all AXI-lite buses and interrupts The frequency is fixed at **50MHz (clock period 20 ns)**
`rst_n`	I	1	**Asynchronous** active low reset signal

Top I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_awaddr`	I	10	Address of the first transfer in a transaction.
`s_axil_awvalid`	I	1	Write address request valid indicator.
`s_axil_awready`	O	1	Write address request ready indicator.
`s_axil_wdata`	I	128	Write data.
`s_axil_wvalid`	I	1	Write data valid indicator.
`s_axil_wready`	O	1	Write data ready indicator
`s_axil_bresp`	O	2	Response for write channel **(Only 2'b00 allowed).**
`s_axil_bvalid`	O	1	Write response valid indicator.
`s_axil_bready`	I	1	Write response ready indicator.

Top I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_araddr`	I	10	Address of the first transfer in a transaction.
`s_axil_arvalid`	I	1	Read address request valid indicator.
`s_axil_arready`	O	1	Read address request ready indicator.
`s_axil_rdata`	O	128	Read data.
`s_axil_rresp`	O	2	Response for read channel **(Only 2'b00 allowed).**
`s_axil_rvalid`	O	1	Read valid indicator.
`s_axil_rready`	I	1	Read ready indicator.

Top I/O: DRAM AXI4 Full

Signal Name	I/O	Width	Description
m_dram_<0-3>_awid	O	4	ID of the write transaction.
m_dram_<0-3>_awaddr	O	32	Address of the first transfer in a transaction.
m_dram_<0-3>_awlen	O	8	Total number of transfers in a transaction
m_dram_<0-3>_awsize	O	3	Maximum number of bytes in a transfer within a transaction.
m_dram_<0-3>_awburst	O	2	Mode of the address increment.
m_dram_<0-3>_awvalid	O	1	Write address request valid indicator.
m_dram_<0-3>_awready	I	1	Write address request ready indicator.
m_dram_<0-3>_wdata	O	128	Write data.
m_dram_<0-3>_wstrb	O	16	Indicator of which byte lanes of write data contain valid data.
m_dram_<0-3>_wlast	O	1	Indicator of the last data.

Top I/O: DRAM AXI4 Full

Signal Name	I/O	Width	Description
`m_dram_<0-3>_wvalid`	O	1	Write data valid indicator.
`m_dram_<0-3>_wready`	I	1	Write data ready indicator
`m_dram_<0-3>_bid`	I	4	ID of the write response transaction.
`m_dram_<0-3>_bresp`	I	2	Response for write channel **(Only 2'b00 allowed)**.
`m_dram_<0-3>_bvalid`	I	1	Write response valid indicator.
`m_dram_<0-3>_bready`	O	1	Write response ready indicator.
`m_dram_<0-3>_arid`	O	4	ID of the read transaction.
`m_dram_<0-3>_araddr`	O	32	Address of the first transfer in a transaction.
`m_dram_<0-3>_arlen`	O	8	Total number of transfers in a transaction
`m_dram_<0-3>_arsize`	O	3	Maximum number of bytes in a transfer within a transaction.

Top I/O: DRAM AXI4 Full

Signal Name	I/O	Width	Description
m_dram_<0-3>_arburst	O	2	Mode of the address increment.
m_dram_<0-3>_arvalid	O	1	Write address request valid indicator.
m_dram_<0-3>_arready	I	1	Write address request ready indicator.
m_dram_<0-3>_rid	I	4	ID of the read transaction.
m_dram_<0-3>_rdata	I	128	Read data.
m_dram_<0-3>_rresp	I	2	Response for read channel **(Only 2'b00 allowed).**
m_dram_<0-3>_rlast	I	1	Indicator of the last data.
m_dram_<0-3>_rvalid	I	1	Read valid indicator.
m_dram_<0-3>_rready	O	1	Read ready indicator.

Controller I/O: Clock, Reset, and Interrupt

Signal Name	I/O	Width	Description
`clk_ctrl_bus`	I	1	The system clock signal for all AXI-lite buses and interrupts The frequency is fixed at **50MHz (clock period 20 ns)**
`rst_n`	I	1	**Asynchronous** active low reset signal
`interrupt_pe`	I	6	Interrupt signal from PEs (under **`clk_ctrl_bus`** domain)

Controller I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_awaddr`	I	10	Address of the first transfer in a transaction.
`s_axil_awvalid`	I	1	Write address request valid indicator.
`s_axil_awready`	O	1	Write address request ready indicator.
`s_axil_wdata`	I	128	Write data.
`s_axil_wvalid`	I	1	Write data valid indicator.
`s_axil_wready`	O	1	Write data ready indicator
`s_axil_bresp`	O	2	Response for write channel **(Only 2'b00 allowed).**
`s_axil_bvalid`	O	1	Write response valid indicator.
`s_axil_bready`	I	1	Write response ready indicator.

Controller I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_araddr`	I	10	Address of the first transfer in a transaction.
`s_axil_arvalid`	I	1	Read address request valid indicator.
`s_axil_arready`	O	1	Read address request ready indicator.
`s_axil_rdata`	O	128	Read data.
`s_axil_rresp`	O	2	Response for read channel **(Only 2'b00 allowed).**
`s_axil_rvalid`	O	1	Read valid indicator.
`s_axil_rready`	I	1	Read ready indicator.

Controller I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`m_axil_<0-1>_<1-3>_awaddr`	O	10	Address of the first transfer in a transaction.
`m_axil_<0-1>_<1-3>_awvalid`	O	1	Write address request valid indicator.
`m_axil_<0-1>_<1-3>_awready`	I	1	Write address request ready indicator.
`m_axil_<0-1>_<1-3>_wdata`	O	128	Write data.
`m_axil_<0-1>_<1-3>_wvalid`	O	1	Write data valid indicator.
`m_axil_<0-1>_<1-3>_wready`	I	1	Write data ready indicator
`m_axil_<0-1>_<1-3>_bresp`	I	2	Response for write channel.
`m_axil_<0-1>_<1-3>_bvalid`	I	1	Write response valid indicator.
`m_axil_<0-1>_<1-3>_bready`	O	1	Write response ready indicator.

Controller I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`m_axil_<0-1>_<1-3>_araddr`	O	10	Address of the first transfer in a transaction.
`m_axil_<0-1>_<1-3>_arvalid`	O	1	Read address request valid indicator.
`m_axil_<0-1>_<1-3>_arready`	I	1	Read address request ready indicator.
`m_axil_<0-1>_<1-3>_rdata`	I	128	Read data.
`m_axil_<0-1>_<1-3>_rresp`	I	2	Response for read channel.
`m_axil_<0-1>_<1-3>_rvalid`	I	1	Read valid indicator.
`m_axil_<0-1>_<1-3>_rready`	O	1	Read ready indicator.

PE I/O: Clock, Reset, and Interrupt

Signal Name	I/O	Width	Description
`clk_pe`	I	1	The system clock signal for PEs You can define the clock frequency (>= 100MHz)
`clk_data_bus`	I	1	The system clock signal for all AXI-full buses The frequency is fixed at **200MHz (clock period 5ns)**
`clk_ctrl_bus`	I	1	The system clock signal for all AXI-lite buses and interrupts The frequency is fixed at **50MHz (clock period 20 ns)**
`rst_n`	I	1	**Asynchronous** active low reset signal
`interrupt_out`	I	1	Interrupt signal from PEs (under **`clk_ctrl_bus`** domain)

PE I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_awaddr`	I	10	Address of the first transfer in a transaction.
`s_axil_awvalid`	I	1	Write address request valid indicator.
`s_axil_awready`	O	1	Write address request ready indicator.
`s_axil_wdata`	I	128	Write data.
`s_axil_wvalid`	I	1	Write data valid indicator.
`s_axil_wready`	O	1	Write data ready indicator
`s_axil_bresp`	O	2	Response for write channel.
`s_axil_bvalid`	O	1	Write response valid indicator.
`s_axil_bready`	I	1	Write response ready indicator.

PE I/O: AXI4 Lite

Signal Name	I/O	Width	Description
`s_axil_araddr`	I	10	Address of the first transfer in a transaction.
`s_axil_arvalid`	I	1	Read address request valid indicator.
`s_axil_arready`	O	1	Read address request ready indicator.
`s_axil_rdata`	O	128	Read data.
`s_axil_rresp`	O	2	Response for read channel.
`s_axil_rvalid`	O	1	Read valid indicator.
`s_axil_rready`	I	1	Read ready indicator.

PE I/O: AXI4 Full

Signal Name	I/O	Width	Description
m_axi_<left, right, or vertical>_awid	O	4	ID of the write transaction.
m_axi_<left, right, or vertical>_awaddr	O	32	Address of the first transfer in a transaction.
m_axi_<left, right, or vertical>_awlen	O	8	Total number of transfers in a transaction
m_axi_<left, right, or vertical>_awsize	O	3	Maximum number of bytes in a transfer within a transaction.
m_axi_<left, right, or vertical>_awburst	O	2	Mode of the address increment.
m_axi_<left, right, or vertical>_awvalid	O	1	Write address request valid indicator.
m_axi_<left, right, or vertical>_awready	I	1	Write address request ready indicator.
m_axi_<left, right, or vertical>_wdata	O	128	Write data.
m_axi_<left, right, or vertical>_wstrb	O	16	Indicator of which byte lanes of write data contain valid data.
m_axi_<left, right, or vertical>_wlast	O	1	Indicator of the last data.

PE I/O: AXI4 Full

Signal Name	I/O	Width	Description
`m_axi_<left, right, or vertical>_wvalid`	O	1	Write data valid indicator.
`m_axi_<left, right, or vertical>_wready`	I	1	Write data ready indicator
`m_axi_<left, right, or vertical>_bid`	I	4	ID of the write response transaction.
`m_axi_<left, right, or vertical>_bresp`	I	2	Response for write channel.
`m_axi_<left, right, or vertical>_bvalid`	I	1	Write response valid indicator.
`m_axi_<left, right, or vertical>_bready`	O	1	Write response ready indicator.
`m_axi_<left, right, or vertical>_arid`	O	4	ID of the read transaction.
`m_axi_<left, right, or vertical>_araddr`	O	32	Address of the first transfer in a transaction.
`m_axi_<left, right, or vertical>_arlen`	O	8	Total number of transfers in a transaction
`m_axi_<left, right, or vertical>_arsize`	O	3	Maximum number of bytes in a transfer within a transaction.

PE I/O: AXI4 Full

Signal Name	I/O	Width	Description
`m_axi_<left, right, or vertical>_arburst`	O	2	Mode of the address increment.
`m_axi_<left, right, or vertical>_arvalid`	O	1	Write address request valid indicator.
`m_axi_<left, right, or vertical>_arready`	I	1	Write address request ready indicator.
`m_axi_<left, right, or vertical>_rid`	I	4	ID of the read transaction.
`m_axi_<left, right, or vertical>_rdata`	I	128	Read data.
`m_axi_<left, right, or vertical>_rresp`	I	2	Response for read channel.
`m_axi_<left, right, or vertical>_rlast`	I	1	Indicator of the last data.
`m_axi_<left, right, or vertical>_rvalid`	I	1	Read valid indicator.
`m_axi_<left, right, or vertical>_rready`	O	1	Read ready indicator.

PE I/O: AXI4 Full

Signal Name	I/O	Width	Description
`s_axi_<left, right, or vertical>_awid`	I	4	ID of the write transaction.
`s_axi_<left, right, or vertical>_awaddr`	I	32	Address of the first transfer in a transaction.
`s_axi_<left, right, or vertical>_awlen`	I	8	Total number of transfers in a transaction
`s_axi_<left, right, or vertical>_awsize`	I	3	Maximum number of bytes in a transfer within a transaction.
`s_axi_<left, right, or vertical>_awburst`	I	2	Mode of the address increment.
`s_axi_<left, right, or vertical>_awvalid`	I	1	Write address request valid indicator.
`s_axi_<left, right, or vertical>_awready`	O	1	Write address request ready indicator.
`s_axi_<left, right, or vertical>_wdata`	I	128	Write data.
`s_axi_<left, right, or vertical>_wstrb`	I	16	Indicator of which byte lanes of write data contain valid data.
`s_axi_<left, right, or vertical>_wlast`	I	1	Indicator of the last data.

PE I/O: AXI4 Full

Signal Name	I/O	Width	Description
`s_axi_<left, right, or vertical>_wvalid`	I	1	Write data valid indicator.
`s_axi_<left, right, or vertical>_wready`	O	1	Write data ready indicator
`s_axi_<left, right, or vertical>_bid`	O	4	ID of the write response transaction.
`s_axi_<left, right, or vertical>_bresp`	O	2	Response for write channel **(Only 2'b00 allowed)**.
`s_axi_<left, right, or vertical>_bvalid`	O	1	Write response valid indicator.
`s_axi_<left, right, or vertical>_bready`	I	1	Write response ready indicator.
`s_axi_<left, right, or vertical>_arid`	I	4	ID of the read transaction.
`s_axi_<left, right, or vertical>_araddr`	I	32	Address of the first transfer in a transaction.
`s_axi_<left, right, or vertical>_arlen`	I	8	Total number of transfers in a transaction
`s_axi_<left, right, or vertical>_arsize`	I	3	Maximum number of bytes in a transfer within a transaction.

Top I/O: AXI4 Full

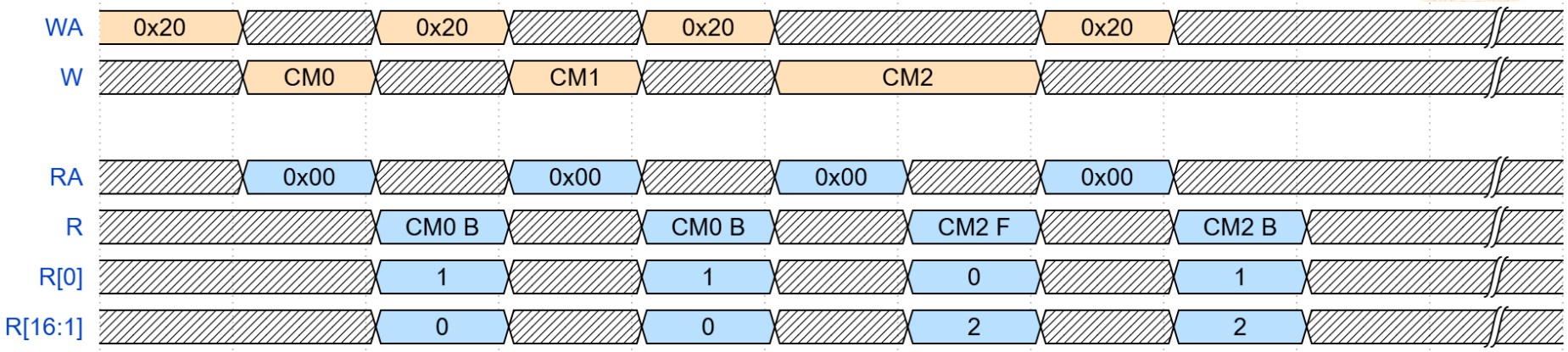
Signal Name	I/O	Width	Description
s_axi_<left, right, or vertical>_arburst	I	2	Mode of the address increment.
s_axi_<left, right, or vertical>_arvalid	I	1	Write address request valid indicator.
s_axi_<left, right, or vertical>_arready	O	1	Write address request ready indicator.
s_axi_<left, right, or vertical>_rid	O	4	ID of the read transaction.
s_axi_<left, right, or vertical>_rdata	O	128	Read data.
s_axi_<left, right, or vertical>_rresp	O	2	Response for read channel.
s_axi_<left, right, or vertical>_rlast	O	1	Indicator of the last data.
s_axi_<left, right, or vertical>_rvalid	O	1	Read valid indicator.
s_axi_<left, right, or vertical>_rready	I	1	Read ready indicator.

Controller Register Map

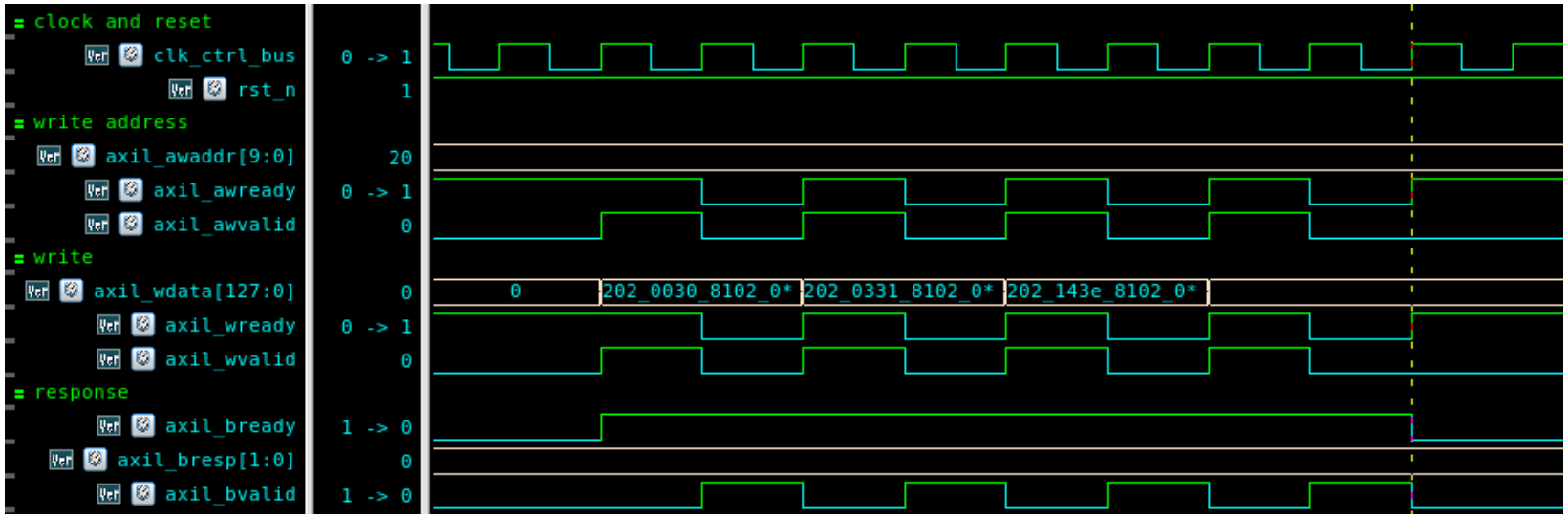
- The operation command is assigned by writing the register with an address and value pair

Address	Access	Data Content (128-bit)	Data Note
0x020	Write Only	bit 7–0: OPCODE, bit 31–8: reserved, bit 63–32: X_META, bit 95–64: Y_META, bit 127–96: Z_META	COMMAND_FIFO: push a 128-bit packed command (STALL or GEMM)
0x010	—	Reserved	Reserved for future use
0x000	Read Only	bit 0: BUSY, bit 16–1: process_counter, bit 127–17: reserved	BUSY=1 during execution; process counter tracks progress

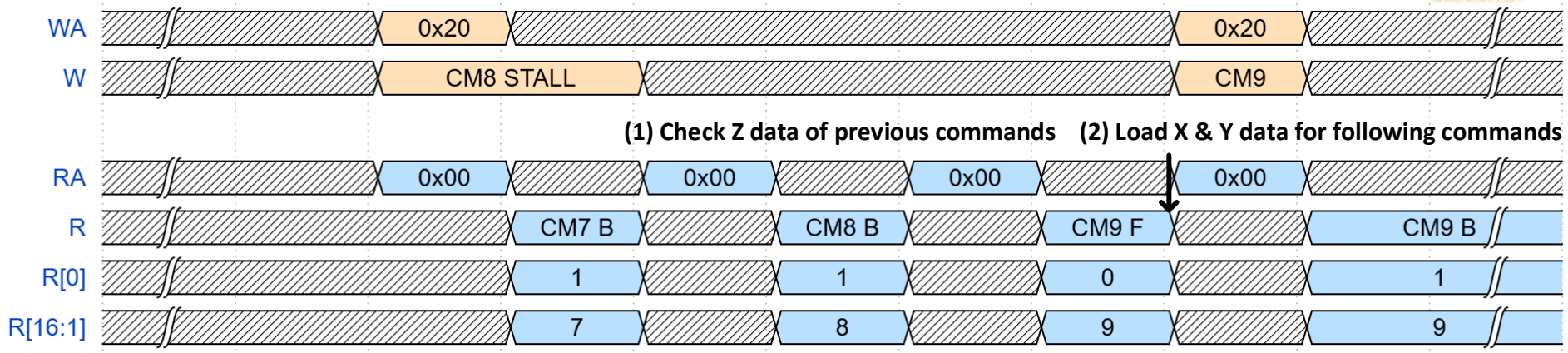
MCU Interaction Example



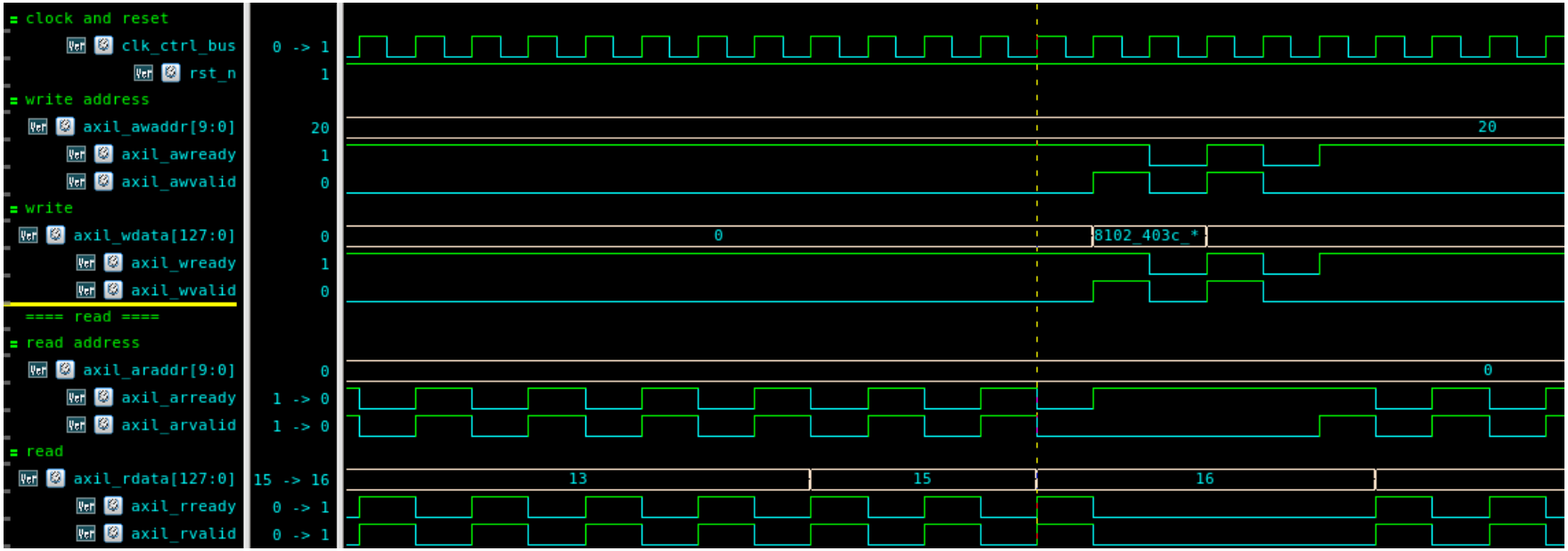
* The B channel is not shown here; * CMn: a command with ID n; * B = Busy, F = Free



MCU Interaction Example



* CMn: a command with ID n; * B = Busy, F = Free



Command Details

- OPCODE

OPCODE	Operation	Description
0x00	STALL	**Sync barrier:** all prior GEMM commands must complete and write back to DRAM before STALL finishes. Acts as a consistency checkpoint.
0x01	GEMM	**Execute** $\boldsymbol{Z} = \boldsymbol{X} \times \boldsymbol{Y}$

Command Details

- META field of $\boldsymbol{X}$, $\boldsymbol{Y^T}$, and $\boldsymbol{Z}$

Bits	Field	Description
`[1:0]`	dtype	`00`=BF16, `01`=FP8-E4M3, `10`=FP8-E5M2
`[5:2]`	bank_mask	DRAM banks **{D, C, B, A}** bitmask. It can be 1, 2, 4 “1” in the field
`[16:6]`	addr_1kb	Base address in units of 1 KB
`[23:17]`	ncol_code	Columns ÷ 256 **(X: dimension K, YT: dimension K, Z: dimension M)**
`[30:24]`	nrow_code	Row count (interpretation depends on bit 31) **(X: dimension N, YT: dimension M, Z: dimension N)**
`[31]`	nrow_divided_by_256	1 = nrow_code is ÷256; 0 = direct count

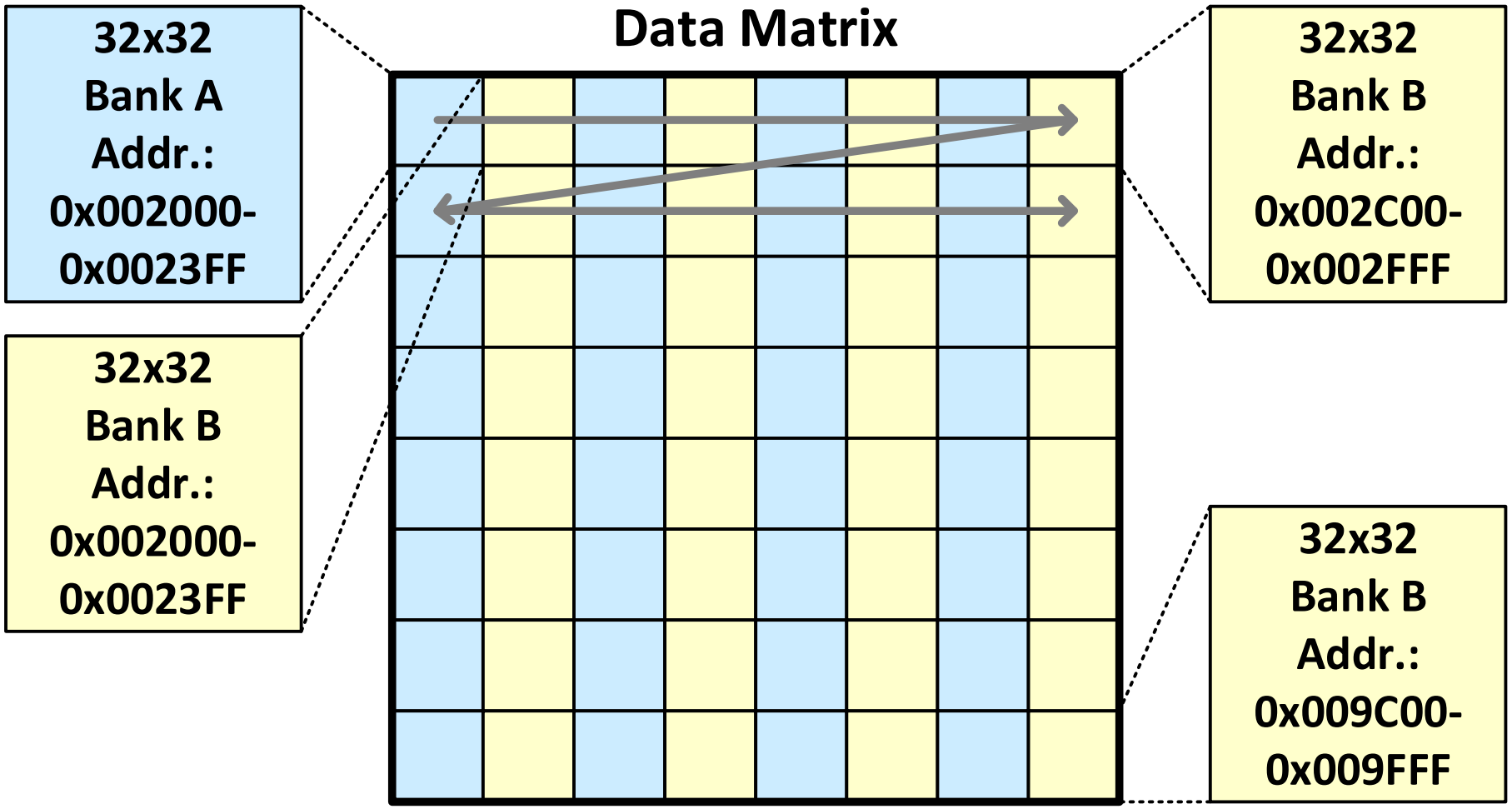
Command Details

- The dimension N can only be 1-127, 512, 768, or 1024
- The dimension K can only be 256, 512, 768, or 1024
- The dimension M can only be 256, 512, 768, or 1024
- Data is stored in **32×32 tile-major** layout in DRAM
  - Each tile contains 32×32 elements stored in row-major order
  - BF16 tile = 2 KB, FP8 tile = 1 KB
- Dimensions not a multiple of 32 are **zero-padded** to the next 32-word boundary

Command Details

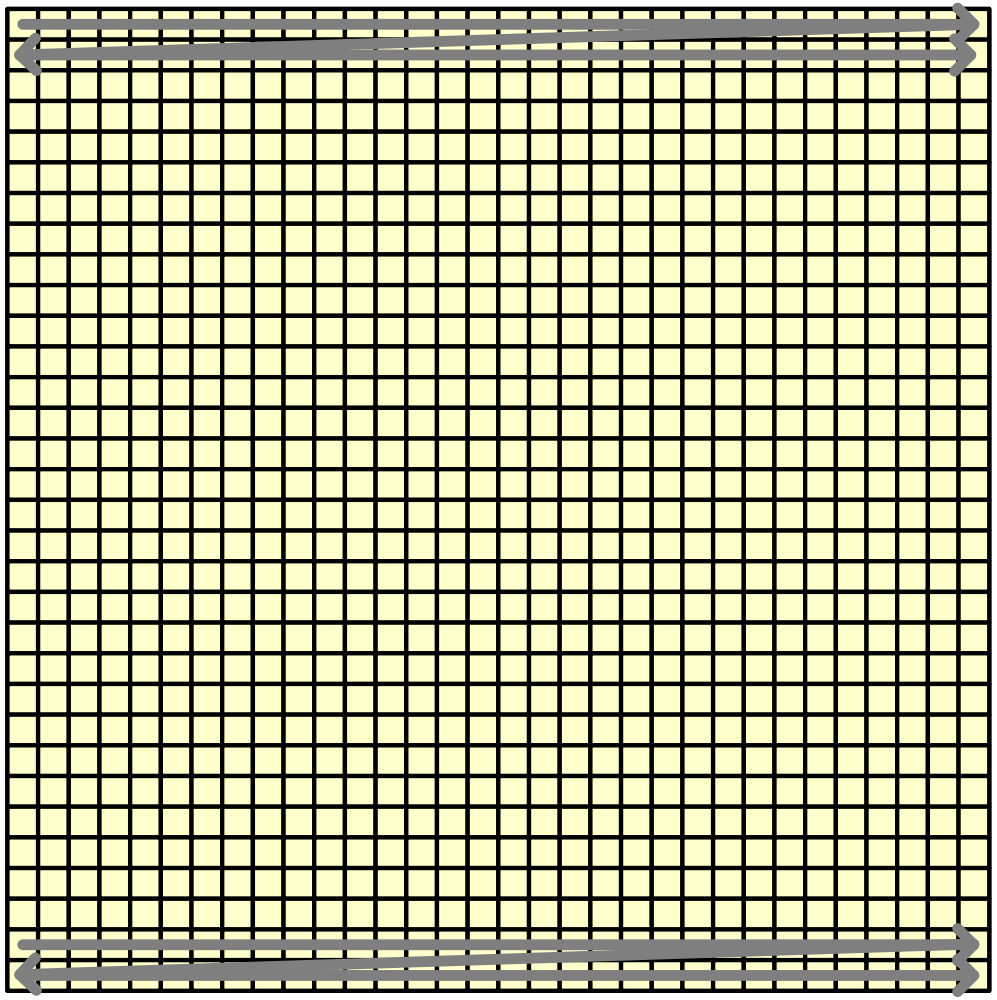
- Case I:
  - [1:0] dtype = 2'b10 (FP8-E5M2)
  - [5:2] bank_mask = 4'b0011 (Bank A + Bank B)
  - [16:6] addr_1kb = 11’d8 (base addr = 8 KB)
  - [23:17] ncol_code = 7’d1 (256 columns)
  - [30:24] nrow_code = 7’d1 (256 rows)
  - [31] nrow_is_div_256 = 1’b1

Command Details



Command Details

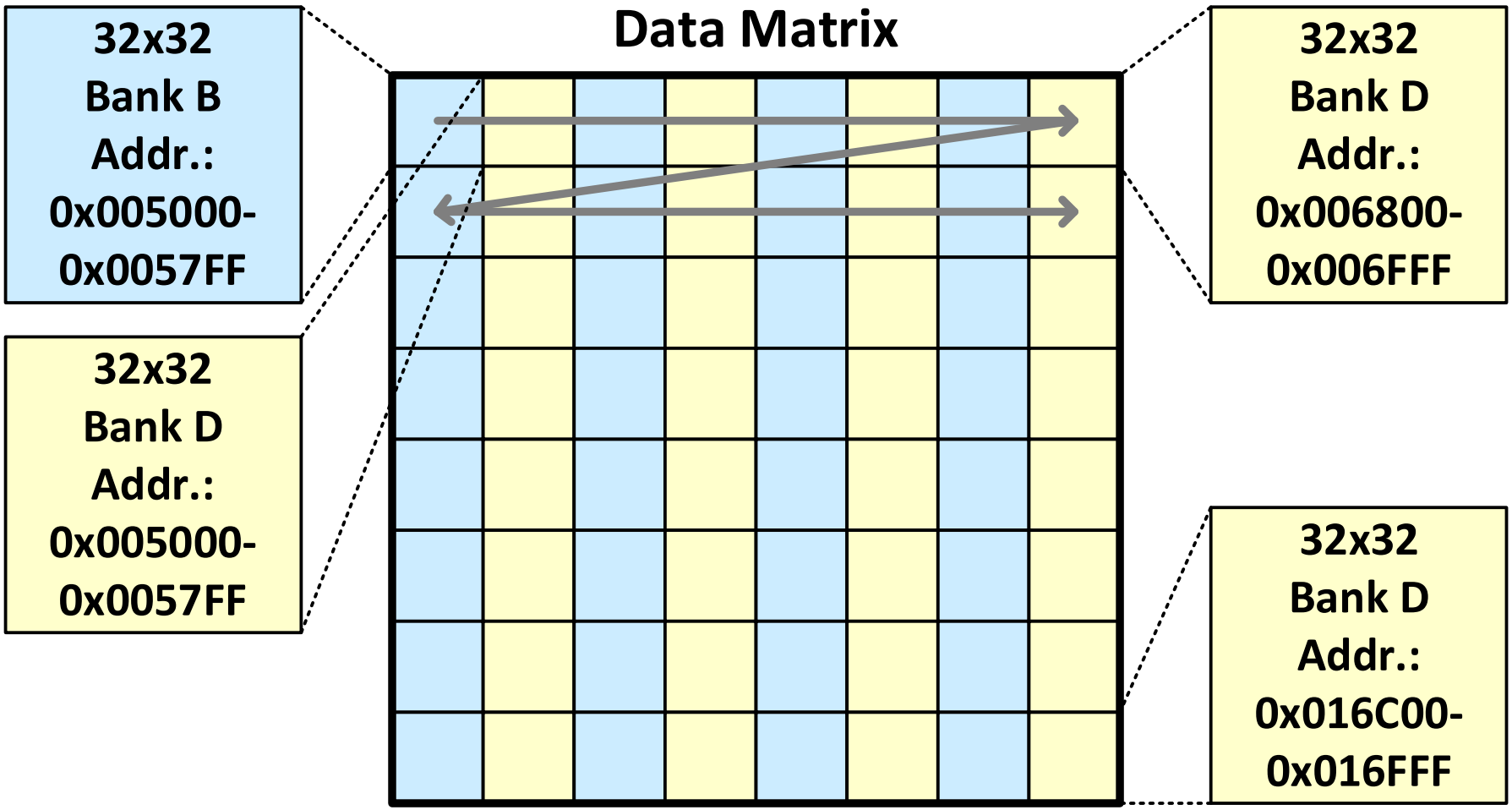
32 x 32 tile

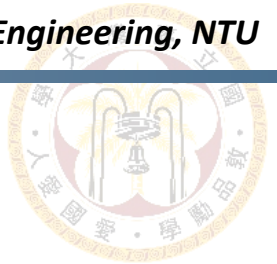


Command Details

- Case II:
  - [1:0] dtype = 2'b00 (BF16)
  - [5:2] bank_mask = 4'b1010 (Bank B + Bank D)
  - [16:6] addr_1kb = 11'd20 (base addr = 20 KB)
  - [23:17] ncol_code = 7'd1 (256 columns)
  - [30:24] nrow_code = 7'd1 (256 rows)
  - [31] nrow_is_div_256 = 1'b1

Command Details

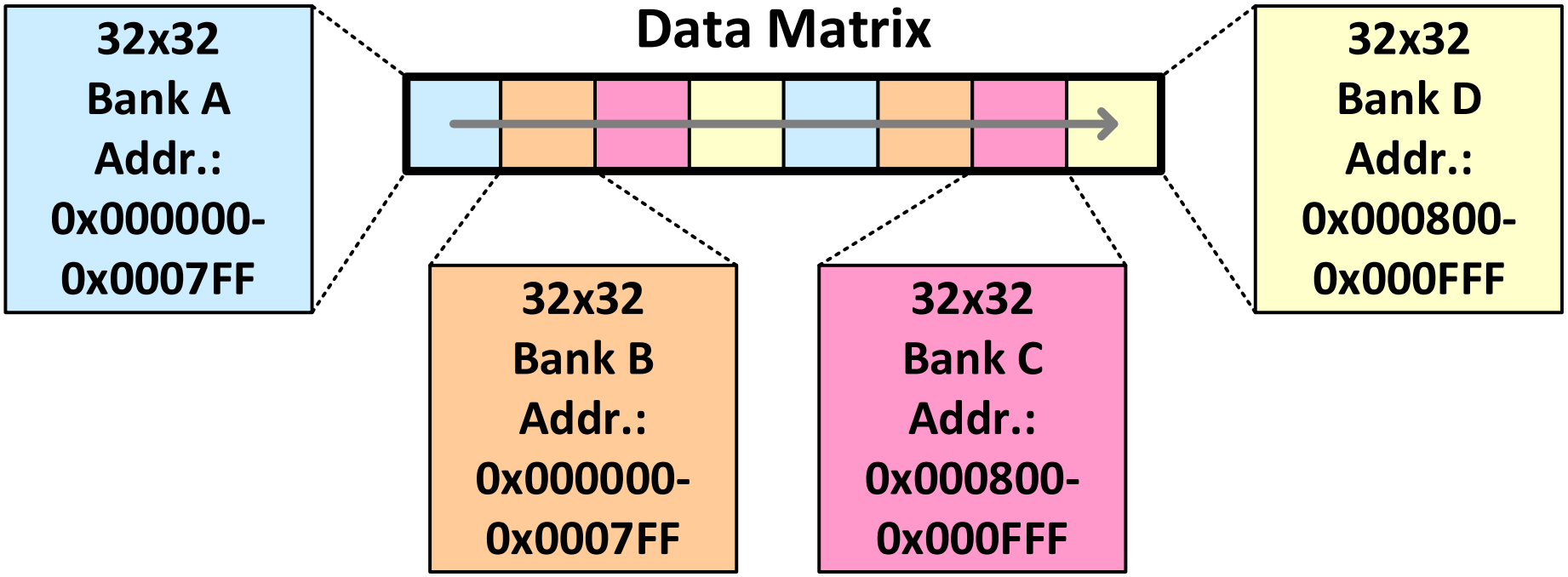




Command Details

- Case III:
  - [1:0] dtype = 2'b00 (BF16)
  - [5:2] bank_mask = 4'b1111
    (Bank A + Bank B + Bank C + Bank D)
  - [16:6] addr_1kb = 11'd0 (base addr = 0 KB)
  - [23:17] ncol_code = 7'd1 (256 columns)
  - [30:24] nrow_code = 7'd2 (2 rows)
  - [31] nrow_is_div_256 = 1'b0

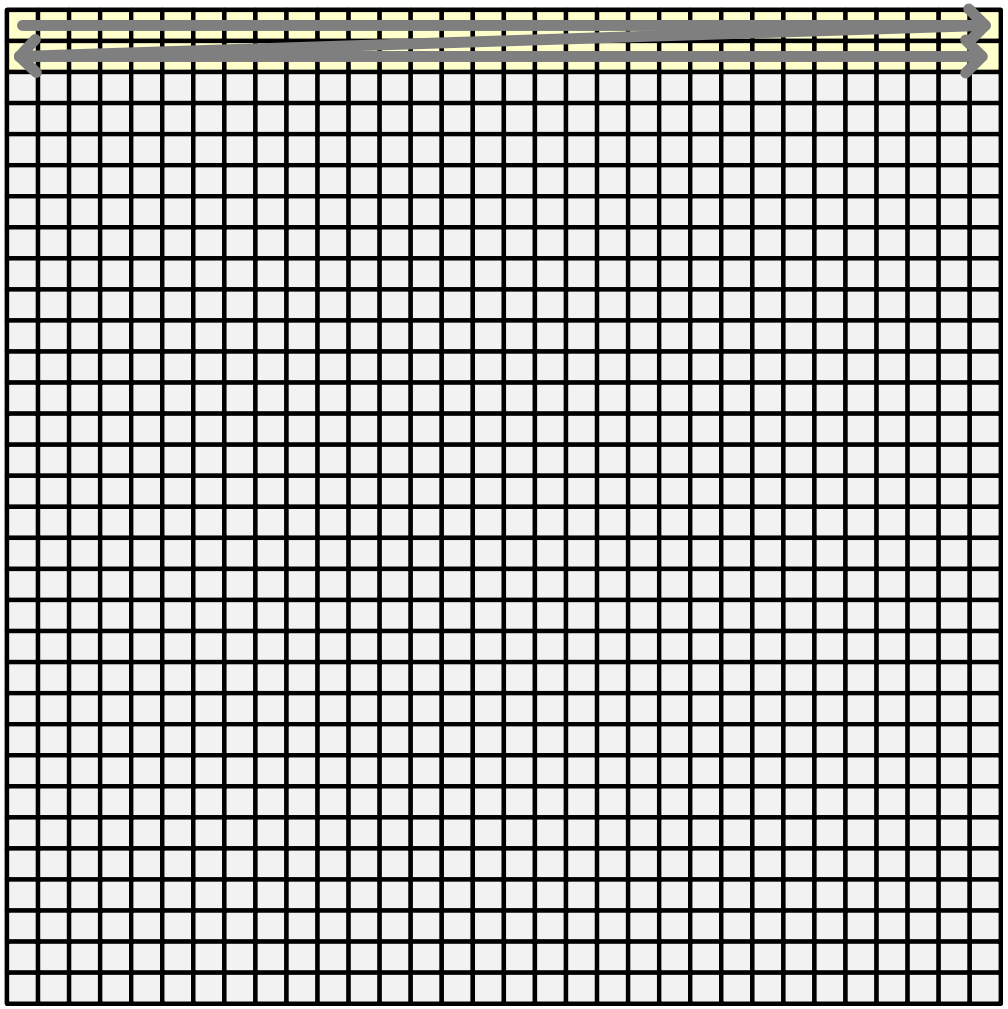
Command Details

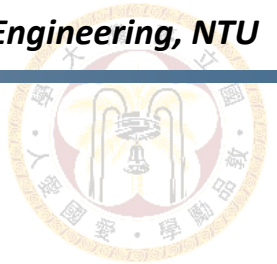


Command Details

32 x 32 tile

Appended Zero





DRAM Data Structure

- Each DRAM contains 4MB of data
  - Total 16MB for 4 DRAMs
- The first **2MB** contains **I/O data**
  - Address range: 0x000000 – 0x1FFFFF
  - The MCU can overwrite the data by X and Y in this range
  - The MCU checks the data of Z in this range
- The other **2MB** contains **temporary data**
  - Address range: 0x200000 – 0x3FFFFF
  - The testbed/MCU won't touch this part
- There will be some latency in read/write operations
  - Read latency: **30 cycles** / Write latency: **20 cycles**

DRAM Data Structure

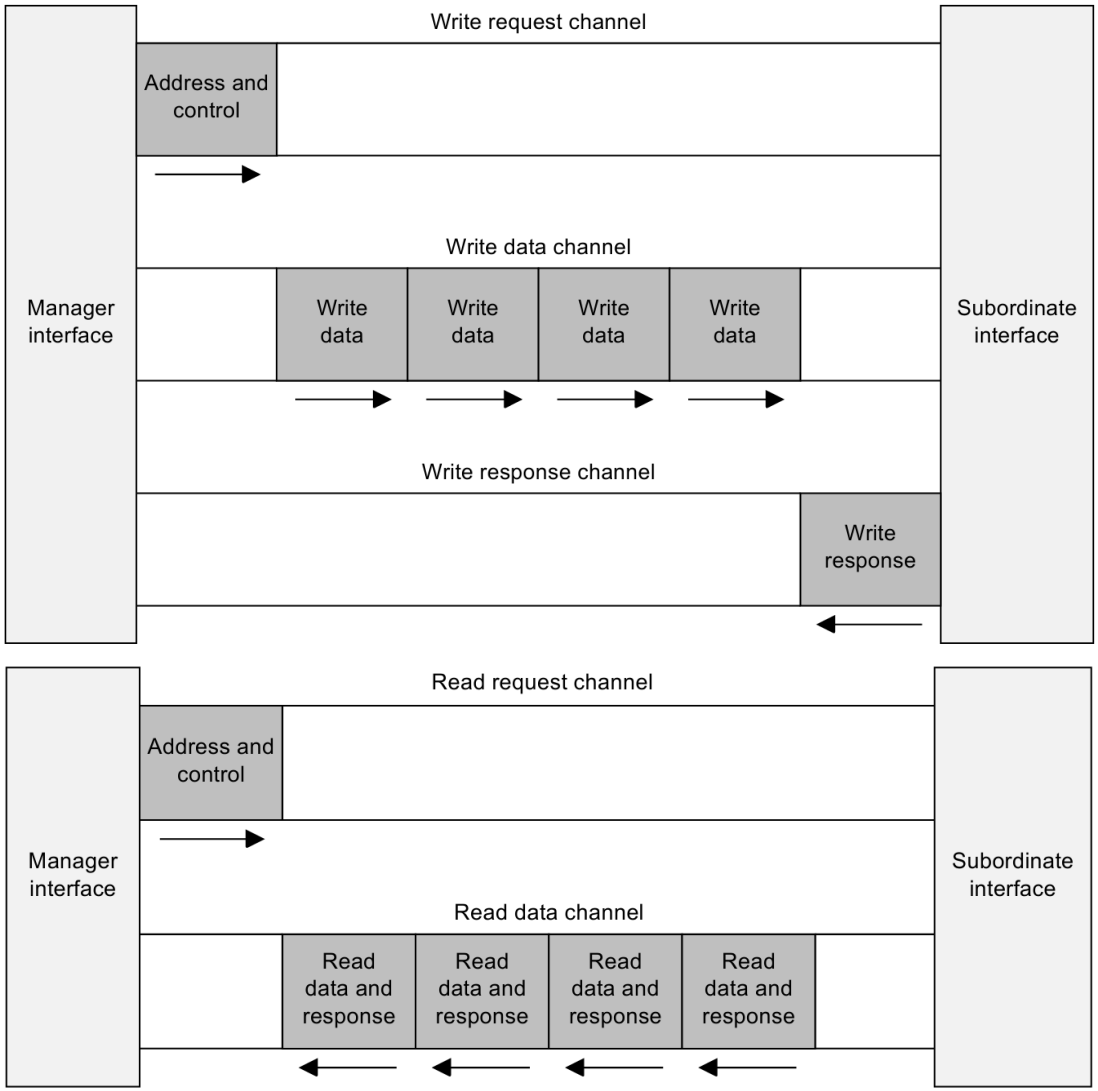
- All data addresses are **aligned to 4 KB**
- Matrices use **32 × 32 block-major (tile-major)** layout, row-major within each tile
- X, Y, Z are distributed across 1–4 DRAM banks based on each command's `bank_mask` field
- Dimensions not a multiple of 32 are **zero-padded** to the next multiple of 32

LOAD and DUMP DRAM Data

- Load data into a specific dram
  - `testbed.dram_gen[0-3].u_dram.load_file(fpath, addr, nwords);`
  - fpath: the path to the file
  - addr: the first position of the data
  - nwords: the number of 4-byte words to be loaded

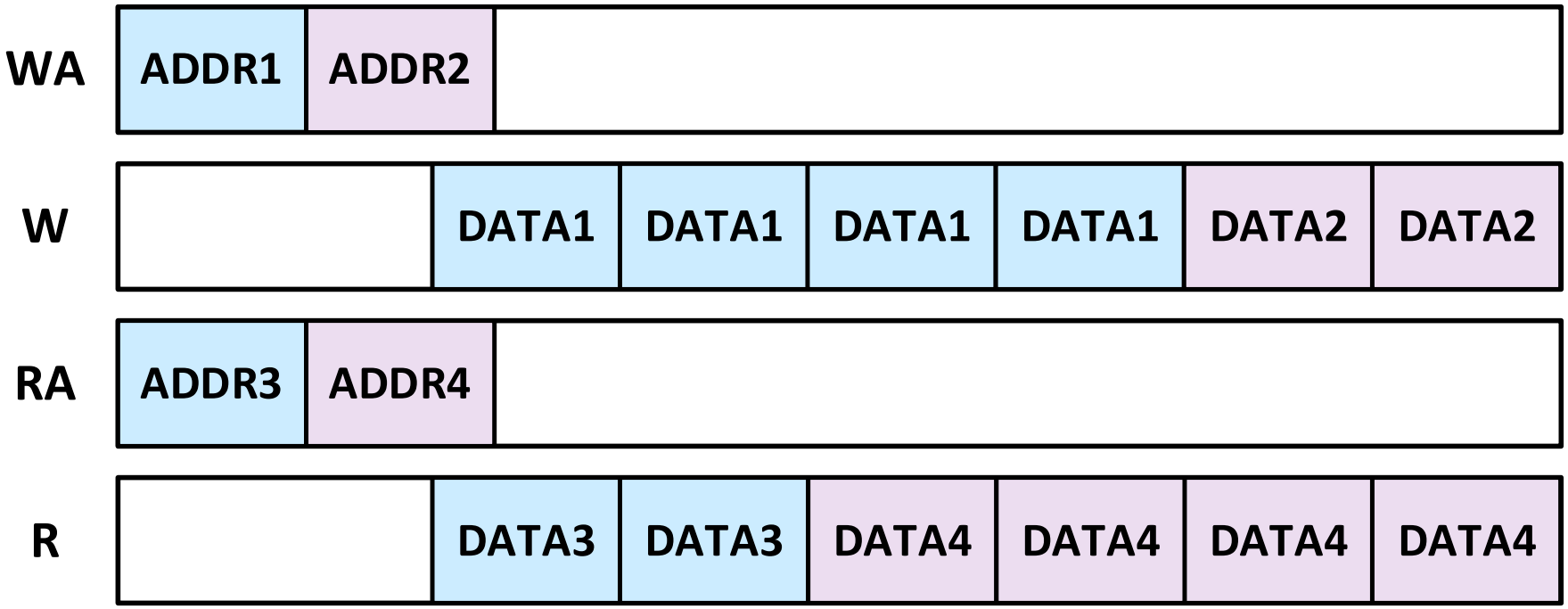
- Dump data from a specific dram
  - `testbed.dram_gen[0-3].u_dram.dump_file(fpath, addr, nwords);`

DRAM Data Transfer



Outstanding Transaction

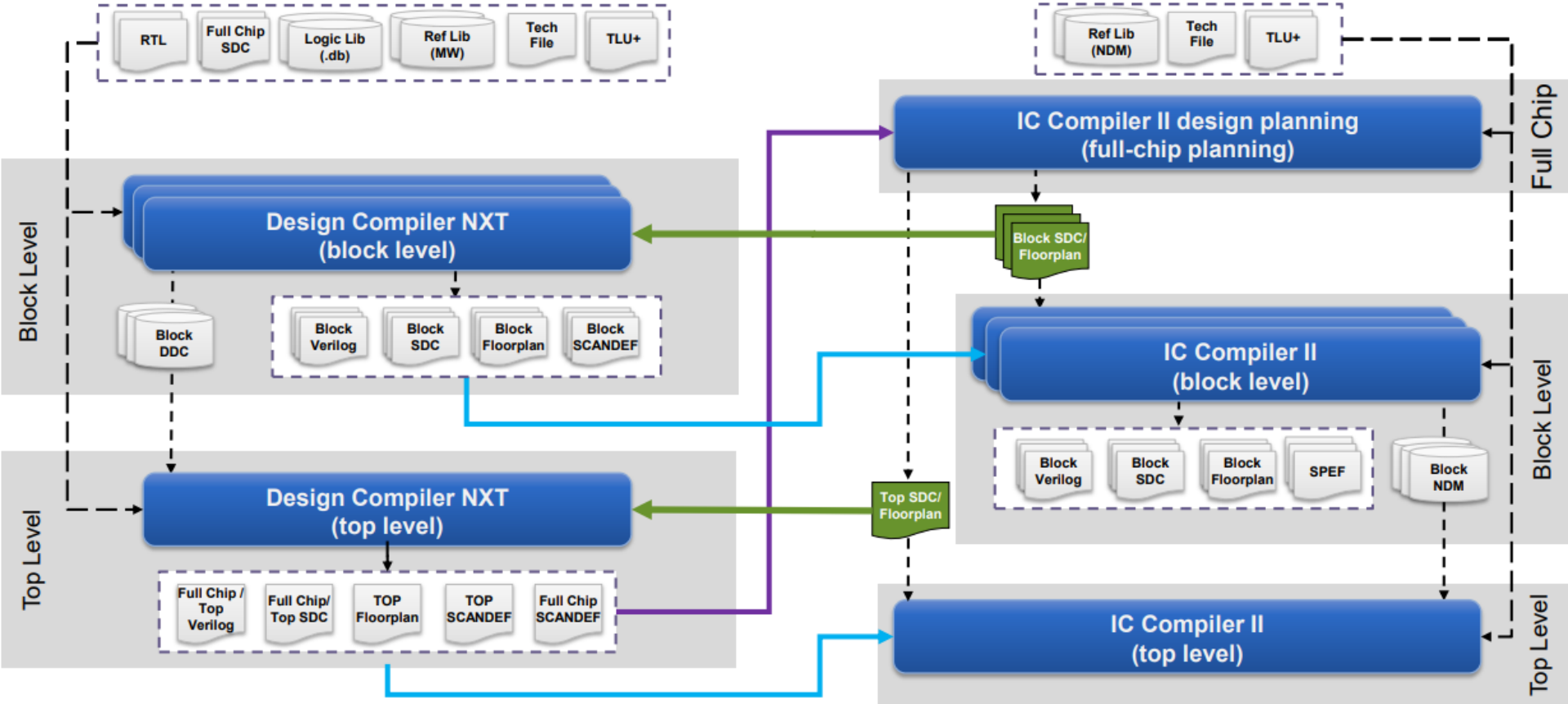
- Outstanding: the new transaction is not to wait until all previous ones are finished
- Total **8** outstanding transactions supported



Process Element

- The result will be slightly different with different accumulation orders in the MAC operation
- Low precision for production; high precision for accumulation
  - **BF16 (E8M7) production**:
    - DW_fp_mult / DW_fp_mult_DG
  - **FP32 (E8M23) accumulation**:
    - DW_fp_add / DW_fp_add_DG
    - Intermediate data should also be FP32
- The RMS error ratio should be less than a certain threshold
  - 5 % for BF16
  - 10 % for FP8-E4M3
  - 15 % for FP8-E5M2

Hierarchical Flow Overview

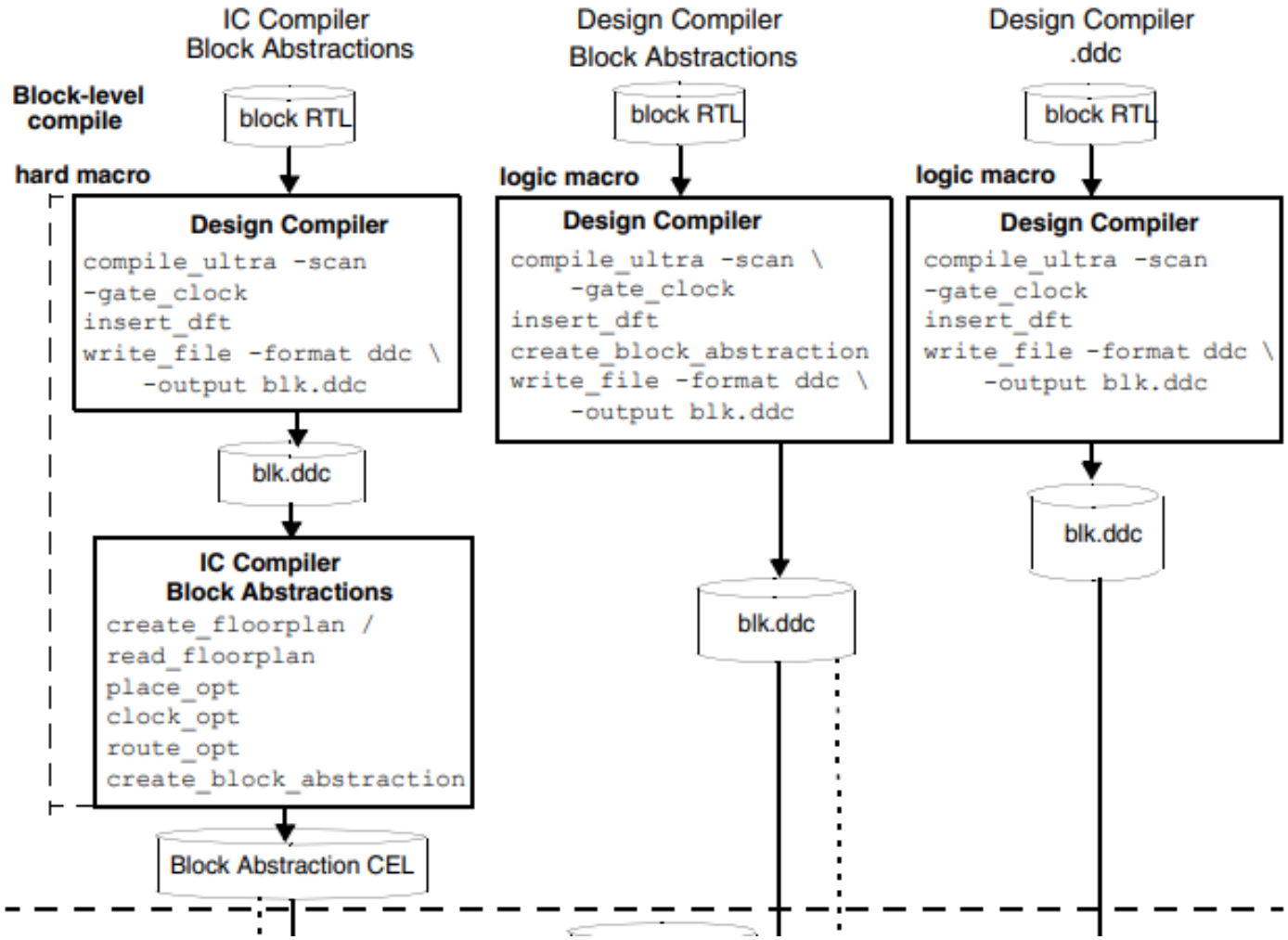


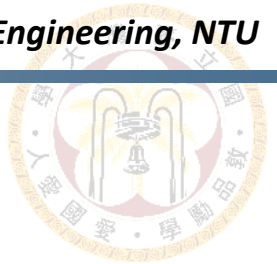
Submodule Synthesis

- Make the design as a block
  - create_block_abstraction
  - Preserve all cells, pins, and nets in timing paths from input ports to registers or output ports
  - Preserve all cells, pins, and nets in timing paths to output ports from registers or input ports

- Write the abstract ddc
  - write_file –format ddc –output pe.abs.ddc

Submodule Synthesis

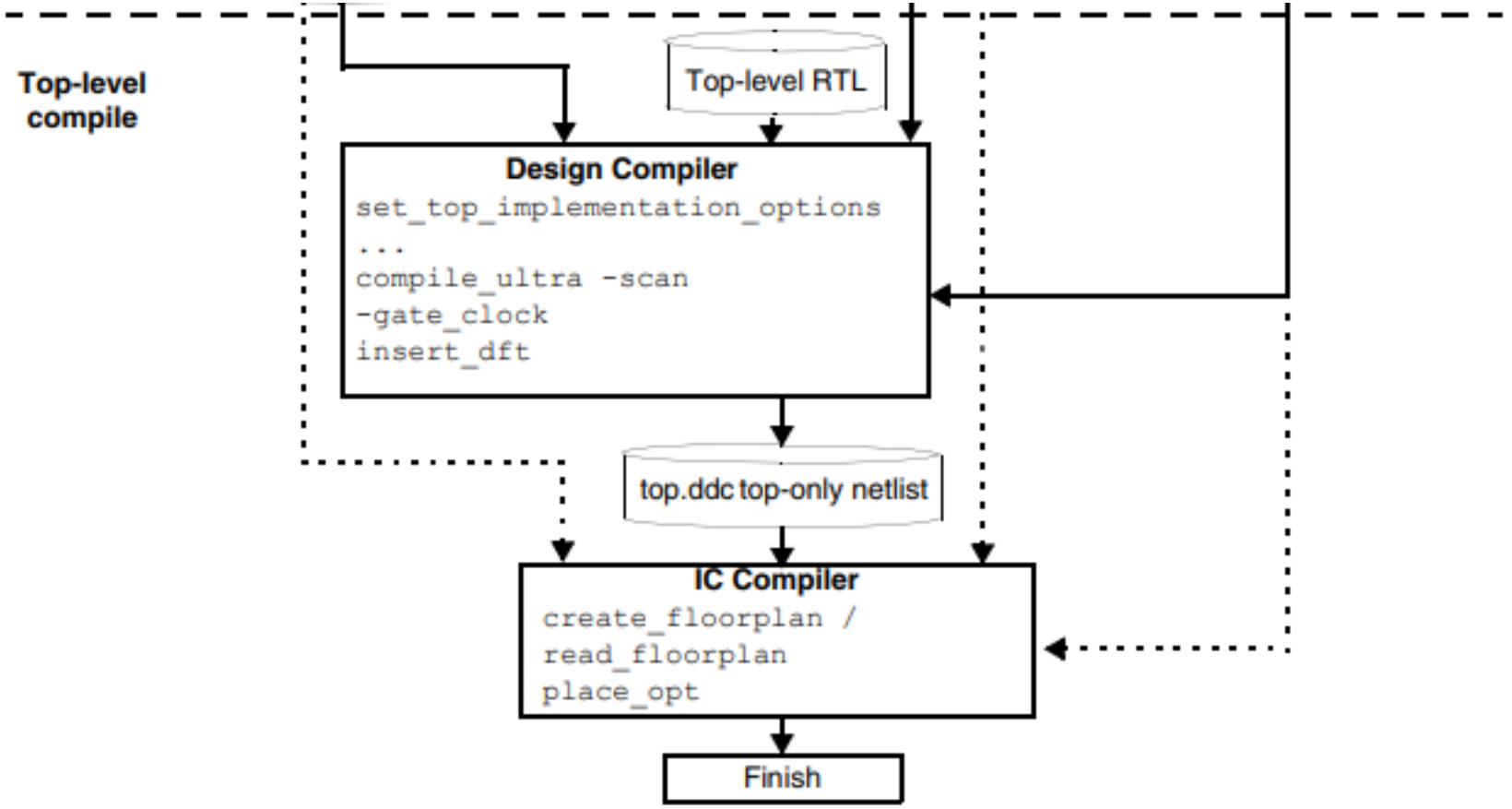




Top Module Synthesis

- Read the abstract submodule ddc
  - read_ddc pe.abs.ddc
- Set hierarchical synthesis option
  - set_top_implementation_options -block_references {pe}

Top Module Synthesis



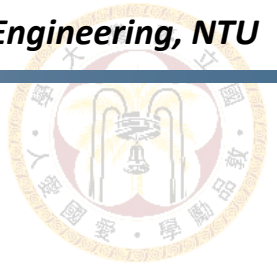
Submodule APR: Specify Used Layers

- The top layers should be preserved for top-level power connection and signal routing
  - `setDesignMode -bottomRoutingLayer 2`
  - `setDesignMode -topRoutingLayer  {your top routing layer in submodule (e.g., 7)}`

Submodule APR: Output Files

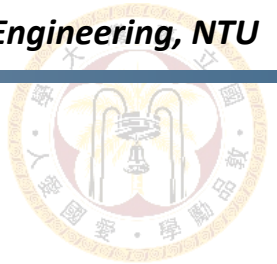
- Export the lef file after APR
  - ```
    write_lef_abstract output/pe_abstract.lef \
        -stripe_pins  -PGPinLayers {Top layers (e.g. M7)}
        -specifyTopLayer {Top stripe layers} -noCutObs
    ```

- Export the lib file
  - ```
    do_extract_model -pg \
        -view    AV_func_ss0p72vm40c \
        -lib_name pe_setup \
        output/pe_apr _ss0p72vm40c.lib
    ```
  - ```
    do_extract_model -pg \
        -view    AV_func_ff0p88v125c \
        -lib_name pe_hold \
        output/pe_apr_ff0p88v125c.lib
    ```
    ```



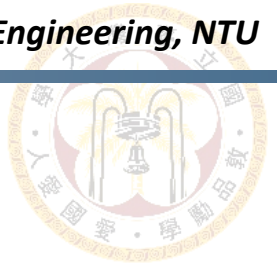
Top Module APR

- Treat the submodule as hard blocks
  - include .lib for timing analysis
  - include .lef for place and route
  - merge .gds when exporting layout file



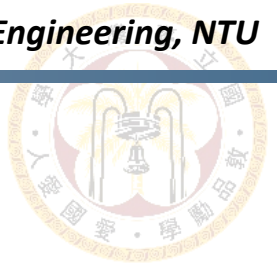
Directory Structure

- 000_tb
  - test_case.csv
  - testbed.sv
  - mcu.sv
  - dram.enc.sv
- 001_interface
  - axi4_full_checker.sv **(TODO)**
  - axi4_lite_checker.sv **(TODO)**
  - axi_if.sv
  - axi_pkg.sv



Directory Structure

- 010_rtl_top
  - top.sv **(TODO)**
  - controller.sv **(TODO)**
- 011_rtl_pe
  - pe.sv **(TODO)**
- 020_rtlsim_top
- 021_rtlsim_pe (optional)
- 030_spyglass_top (optional)
- 031_spyglass_pe (optional)
- 040_syn_top
- 041_syn_pe
- 050_gatesim_top

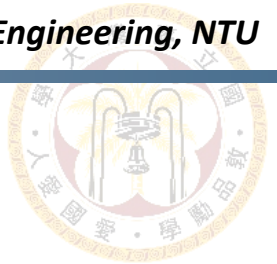


Directory Structure

- 051_gatesim_pe (optional)
- 060_gatesim_power_top
- 061_gatesim_power_pe (optional)
- 070_apr_top
- 071_apr_pe
- 080_postsim_top
- 081_postsim_pe (optional)
- 090_postsim_power_top
- 100_timing_signoff_top
- 110_lvs_top
- 111_lvs_pe
- 120_report

Grading Policy

Item	%	Description
RTL Simulation	20	Pass all test cases
Synthesis	10	5 points for **flat synthesis** and 10 points for **hierarchical synthesis**
APR	10	5 points for **flat APR** and 10 points for **hierarchical APR**
IR Drop	5	The IR drop of the layout should be less than **5%**
Signoff	5	Pass the primetime analysis and LVS check **(2.5points for each)** (DRC not required)
Scan Chain	5	Correct scan chain insertion
AXI Assertion	5	Correct assertion for checking all AXI properties.
Performance	30	Linearly distributed.
Presentation	5	
Report	5	



Simulation Specifications

- RTL simulation should pass all test cases
  - 5% tolerance
  - **BF16** multipliers and **FP32** adders only
  - Hidden test cases included
- Gate-level and post-APR simulation should pass public test cases
  - The result should be the same as the RTL simulation

Synthesis Specifications

- Only the worst-case library is used for synthesis
  - 0p72vm40c
- The slack for setup and hold time should be non-negative
- I/O latency: $PERIOD * 0.5
- Clock uncertainty
  - Setup: 0.2 ns
  - Hold: 0.05 ns
- Only the worst-case library is used in power analysis
  - 0p88v125c

APR Specifications

- Only the **macro layout** is needed
  - IO pads and bonding pads are not required
- **VDD** and **VSS power rings** should each be **2 µm wide**, with **5 ring pairs** required in the top module
- Remember to add the **Power Rail** (follow pin)
- **Dummy metal layers** are not needed
- **Core Filler** must be added
- The **GDSII** file after APR must be generated
- Ensure that the APR DRC/LVS is completely **error-free** (Signoff DRC check is not needed)
- You can generate the IOC file first and then reload the file to set the pin position

Power/IR Drop/LVS Specifications

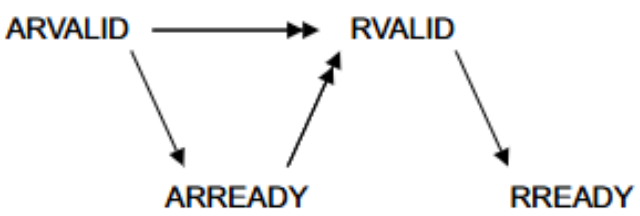
- **Switching activity (SAIF) of public test cases** is utilized to analyze power and simulation
- The IR drop of static power analysis should be less than or equal to **5%**
- The LVS check should be passed, and the post APR simulation should be correct

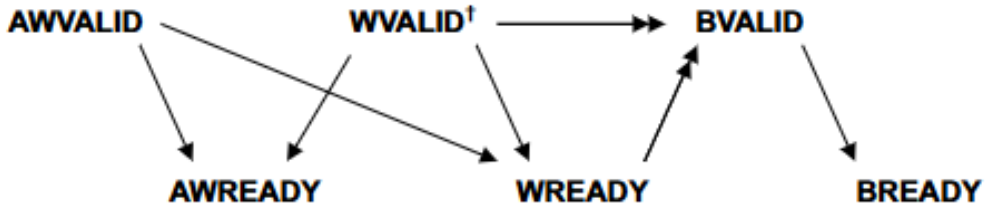
Scan Chain Specifications

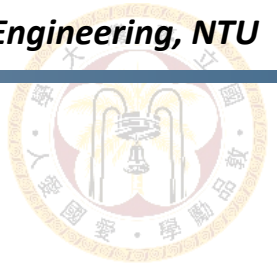
- Three scan chains required
  - SE, SI0, SI1, SI2, SO0, SO1, SO2
- The scan chain should be included in ARP
- The .def of the scan chain is required

AXI Full/Lite Assertion Specifications

- 5 specs:
  - Valid signals cannot drop unless accepted (ready asserted)
  - Data/Address cannot be changed until the handshake
  - The size of the bus should be equal to or less than 128 (AXI full only)
  - WLAST should be asserted at the last data (AXI full only)
  - The order of the transaction should be correct
- The interactions of the internal buffer should also follow these specs

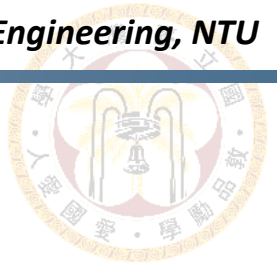






Other Specifications

- Max latency: 150ms
- Max area: 5,000,000 um^2
- Only 16 CPU cores and 48 GB RAM are available for each team
  - The computational resource will be adjusted at 4/17 12:00



Performance Index

The performance index is (Latency^2) * Area * Power

- **For no scan chain insertion: Area * 1.2**

1. **Pass level: A (Hierarchical APR)**
   - Latency: Total latency of the public test cases
   - Area: The area of the top APR
   - Power: The power of the post layout simulation
2. **Pass level: B (Flattened APR)**
   - Latency: Total latency of the public test cases
   - Area: The area of the top APR + 6 * the area of PE APR
   - Power: The power of the post layout simulation

Performance Index

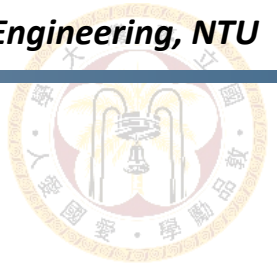
3. Pass level: C (Hierarchical Synthesis)

- Latency: Total latency of the public test cases
- Area: (The area of the top syn. + 6 * the area of PE syn.) * 10
- Power: The power of the gate-level simulation * 2
- Performance score discount 50%

4. Pass level: D (Flattened Synthesis)

- Latency: Total latency of the public test cases
- Area: (The area of the top syn. + 6 * the area of PE syn.) * 10
- Power: The power of the gate-level simulation * 2
- Performance score discount 50%

5. Pass level: E (Behavior)



Review Slide Requirements

1. **The Data Access Scheme of Your Design**
2. **Hardware Implementation**
   - The block diagram of your design
   - The hardware design skills you are going to apply
3. **Other Design Skills**
   - Parallel processing
4. **Performance Estimated**
   - Area/Time/Power estimated based on your design
   - The improvement of your design

Review Slide Requirements

5. **The timeline for implementing your final project**

6. **Task Assignment for the Final Project of your team**

- **Submit the slide to NTU COOL before 12:00, May 12, 2026**
  - Both members in a team need to submit

Report Requirements

1. **APR Results**
   - Show the screenshot of the layout and DRC/LVS results
   - Show the screenshot of the IR drop analysis results
   - Show the screenshot of the clock tree
2. **The LVS Pass screenshot**
3. **Hardware Implementation**
   - Provide specifics about the hardware architecture, including the design of key modules and optimization techniques
4. **Techniques you adopt**

Design Spec File

- Please fill in your design spec
  - Pass level
  - PE clock period (in ns)
  - Total simulation latency (in ns)
  - Top synthesis area: the reported area from the top synthesis
  - PE synthesis area: the reported area from the PE synthesis
  - Top APR area: the reported area from the top APR
  - PE APR area: the reported area from the PE APR
  - Successful Scan Chain Insertion: Yes/No

Final Project Schedule

- **Final Project Announcement:** April 14, 2026
- **Review Slide Submission: 12:00, May 12, 2026**
  - Each team should prepare a 10-minute presentation
- **Final Project Review:** May 12, 2026
- **Due: 14:00, June 9, 2026**
  - **No late submission is allowed**
- **Final Project Presentation:** June 16, 2026
  - Top-performing teams will be invited to present their design optimization strategies
  - Bonus points will be awarded for presentations

Submission – Review Presentation

- Name the presentation slide to **acvsd-hw-teamID-final-v*k*.pdf** (e.g., acvsd-hw-team1-final-v1.pdf)

- Submit to NTU COOL
  - Both members of a team should hand it out to NTU COOL

- **Deadline:** 2026/5/12 12:00:00 (UTC+8)

Submission - Workstation

- Create a folder named **acvsd-hw-teamID-final (in lowercase)** (e.g., acvsd-hw-team1-final-v1) with the structure below:

010_rtl_top
- top.sv
- controller.sv
- [any other design file]
- file.f
- *_checker.sv

011_rtl_pe
- pe.sv
- [any other design file]
- file.f

040_syn_top
- syn.tcl
- syn.sdc
- netlist
  - top_syn.ddc
  - top_syn.sdc
  - top_syn.v
  - top_syn.sdf
  - top_scan.def
- reports
  - top_area.rpt
  - top_timing.rpt

Submission - Workstation

- Create a folder named **acvsd-hw-teamID-final (in lowercase)** (e.g., acvsd-hw-team1-final-v1) with the structure below:

040_syn_pe
- syn.tcl
- syn.sdc
- netlist
  - pe_syn.ddc
  - pe_syn.sdc
  - pe_syn.v
  - pe_syn.sdf
  - pe_scan.def
- reports
  - pe_area.rpt
  - pe_timing.rpt

060_gatesim_power_top
- parallel_run/part*/power.rpt

070_apr_top
- dbs
  - finish.enc
  - finish.enc.dat
- output
  - top_apr.v
  - top_apr.sdf
  - top_apr.gds
  - top_apr_scan.def

Submission - Workstation

- Create a folder named **acvsd-hw-teamID-final (in lowercase)** (e.g., acvsd-hw-team1-final-v1) with the structure below:

071_apr_pe
- dbs
  - finish.enc
  - finish.enc.dat
- output
  - pe_apr.v
  - pe_apr.sdf
  - pe_apr.gds
  - pe_apr _ss0p72vm40c.lib
  - pe_apr _ff0p88v125c.lib
  - pe_apr_scan.def

090_postsim_power_top
- parallel_run/part*/power.rpt

100_timging_signoff_top
- timing_signoff.tcl
- timing.rpt

Submission - Workstation

- Create a folder named **acvsd-hw-teamID-final (in lowercase)** (e.g., acvsd-hw-team1-final) with the structure below:

110_lvs_top
- lvs.rpt

111_lvs_pe
- lvs.rpt

120_report
- report.txt

Submission - Workstation

- Pack the folder **acvsd-hw-teamID-final** into a tar.gz file named **acvsd-hw-teamID-final.tar.gz**(e.g., acvsd-hw-team1-final.tar.gz)
  - `tar -zcvf acvsd-hw-teamID-final.tar.gz acvsd-hw-teamID-final`
  - All letters are in lowercase.
  - Put the tar.gz file at the root of both your ADFP accounts

- TA will only check the latest version

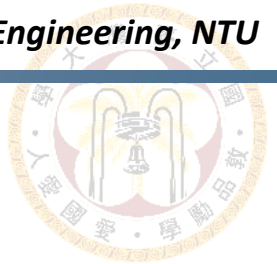
- **Deadline:** 2026/6/9 14:00:00 (UTC+8)

Submission – NTU Cool

- Create a folder named **acvsd-hw-teamID-final (in lowercase)** (EX: **acvsd-hw-teamID-final**) with the structure below:
  - **acvsd-hw-teamID-final**
    - top.sv
    - controller.sv
    - pe.sv
    - [all your design].sv
    - report.pdf
- Compress the folder **acvsd-hw-teamID-final** in a tar file named **acvsd-hw-teamID-final.tar.gz** (e.g., acvsd-hw-team1-final.tar.gz)
  - Both members of a team should hand it out to NTU COOL
- **Deadline:** 2026/6/9 14:00:00 (UTC+8)

Grading Policy

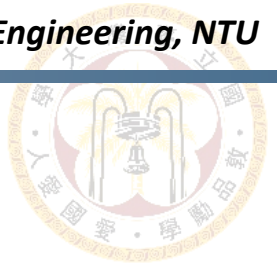
- **No late submission is allowed**
  - Any submission after the deadline will receive 0 points
- **5-point deduction** for incorrect naming or format
  - Pack all files into a single folder and compress the folder
  - Ensure that the files submitted can be decompressed and executed without issues
- **No plagiarism**
  - Plagiarism in any form, including copying from online sources, is strictly prohibited



Discussion

- **NTU Cool Discussion Forum**
  - For any questions not related to assignment answers or privacy concerns, please use the NTU Cool discussion forum
  - TAs will prioritize answering questions on the NTU Cool discussion forum

- **Email:** f12943014@ntu.edu.tw
  - Title should start with [114-2 ACVSD Final Project]
  - Email with a wrong title will be moved to the trash automatically



Reference

- [1] https://docs.nvidia.com/deeplearning/performance/dl-performance-fully-connected/index.html
- [2] https://www.engineered-mind.com/fem/the-finite-element-method-derivation-discrete-system-1/